# Carpe diem (Felix's blog)

## I am a happy developer

- RSS

Search

Navigate...

- Blog
- Archives
- About Me
- Github

# Writing 64 Bit Assembly on Mac OS X

Dec 2nd, 2014

Many assembly tutorials and books doesn't cover how to write a simple assembly program on the Mac OS X. Here are some baby steps that can help people who are also interested in assembly to get started easier.

## Mach-O file format

To get started on writing OSX assembly, you need to understand OSX executable file format – the Mach-O file format. It's similar to ELF, but instead of sections of data, bss, and text, it has segments that contains sections.

A common assembly in Linux like

```
.section data
.section text
# your code here
```

would translate into this in Mach-O

```
.section __DATA,__data
# __DATA is segment, __data is section
.section __TEXT,__text
# __TEXT is segment, __text is section
```

```

# your code here
```

Mach-O is pretty flexible. You can embed a `cstring` section in your `__TEXT` segment instead of putting it in `__DATA,__data`. Actually this is the default behavior that compiler does on your Mac.

# Hello Assembly

Now we know how to translate common linux assembly to mac, let's write a basic program – do a system call with an exit code.

On x86 you do a system call by `int x80` instruction. On 64 bit machine, you do this by `syscall`. Here's the sample code:

```
# Simple exit program
.section __TEXT,__text
.globl _main
_main:
  movl $0x2000001, %eax # system call $1 with $0x2000000 offset
  movl $0, %ebx         # set the exit code to be $0
  syscall
```

you can compile the code by the following commands:

```
as exit.s -o exit.o
ld exit.o -e _main -o exit     # -e specify the entry point of the executable
./exit
echo $? # show last return code, should show 0
```

To perform a system call, you put the system call number in `%eax`, and put the actual exit code to `%ebx`. The system call number can be found in `/usr/include/sys/syscall.h`.

```
#define SYS_syscall        0
#define SYS_exit           1
#define SYS_fork           2
#define SYS_read           3
#define SYS_write          4
#define SYS_open           5
#define SYS_close          6
#define SYS_wait4          7
```

The system call number need to add an offset `0x2000000`, because OSX has 4 different class of system calls. You can find the reference here [XNU syscall](#).

# System call by using wrapper functions

If you're like me that had no assembly background, you might feel that `syscall` is alien to you. In C, we usually use wrapper functions to perform the call:

```
# exit2.s
# To compile it, type the following in shell
# as exit2.s -o exit2.o
# ld exit2.o -e _main -lc -o exit

.section __TEXT,__text
.globl _main
_main:
  pushq %rbp
  movq %rsp, %rbp
  movl $5, %edi        # exit(5);
  callq _exit
```

Now we call a `libc` function instead of performing a system call. To do this we need to link to libc by passing `-lc` to linker `ld`. There are several things you need to do to make a function call.

## Call frame

We need to prepare the stack before we call a function. Else you would probably get a segmentation fault. The values in `%rsp` and `%rbp` is used to preserve frame information. To maintain the stack, you first push the base register `%rbp` onto the stack by `pushq %rbp`; then you copy the stack register `%rsp` to the base register.

If you have local variables, you subtract `%rsp` for space. Remember, stack grows down and heap grows up. When releasing the frame, you add the space back to `%rsp`.

A live cycle of a function would look like this:

```
# function start
pushq %rbp
movq %rsp, %rbp
subq $4, %rsp      # reserve 4 bytes on stack
movl $5, -4(%rbp)  # We usually use base register instead
                   # of stack pointer to access data
addq $4, %rsp      # release the stack
popq %rbp          # restore old %rbp
retq               # function ends
```

The stack size can be set at link time. On OSX, below are the example parameters you can pass to `ld` to set the stack size:

```
ld stack.o -stack_size 0x4000 -stack_addr 0x7fff5fbff000 -lc -e _start -o stack
```

When setting the stack size, you also have to set the stack address. On the System V Application Binary Interface it says

> Although the AMD64 architecture uses 64-bit pointers, implementations are only required to handle 48-bit addresses. Therefore, conforming processes may only use addresses from `0x00000000 00000000` to `0x00007fff ffffffff`

I don't know a good answer of how to chose a good stack address. I just copy whatever a normal code produces.

### Parameters passing

The rules for parameter passing can be found in System V Application Binary Interface:

1. If the class is MEMORY, pass the argument on the stack. If the size of an object is larger than four eight bytes, or it contains unaligned fields, it has class MEMORY.
2. If the class is INTEGER, the next available register of the sequence `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` and `%r9` is used.
3. If the class is SSE, the next available vector register is used, the registers are taken in the order from `%xmm0` to `%xmm7`.

The `exit()` function only need one integer parameter, therefore we put the exit code in `%edi`. Since the parameter is type `int`, we use 32 bit variance of register `%rdi` and the instruction is `movl` (mov long) instead of `movq` (mov quad).

## Hello world

Now we know the basics of how to perform a system call, and how to call a function. Let's write a hello world program.

```
# hello_asm.s
# as hello_asm.s -o hello_asm.o
# ld hello_asm.o -e _main -o hello_asm
.section __DATA,__data
str:
  .asciz "Hello world!\n"

.section __TEXT,__text
.globl _main
_main:
  movl $0x2000004, %eax           # preparing system call 4
  movl $1, %edi                    # STDOUT file descriptor is 1
  movq str@GOTPCREL(%rip), %rsi   # The value to print
  movq $100, %rdx                 # the size of the value to print
  syscall

  movl $0, %ebx
  movl $0x2000001, %eax           # exit 0
```

```
syscall
```

The global variable `str` can only be accessed through GOT (Global Offset Table). And the GOT needs to be access from the instruction pointer `%rip`. For more curious you can read [Mach-O Programming Topics: x86-64 Code Model].

The register used for `syscall` parameters are a little bit different than the normal function call. It uses `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8` and `%r9`. You cannot pass more than 6 parameters in `syscall`, nor can you put the parameters on the stack.

# Hello world using printf

Now you know the basics of assembly. A hello world example using printf should be trivial to read:

```
# hello_asm2.s
# as hello_asm2.s -o hello_asm2.o
# ld hello_asm2.o -e _main -lc -o hello_asm2
.section __DATA,__data
str:
  .asciz "Hello world!\n"

.section __TEXT,__text
.globl _main
_main:
  pushq %rbp
  movq %rsp, %rbp
  movq str@GOTPCREL(%rip), %rdi
  movb $0, %al
  callq _printf
  popq %rbp
  movl $0x2000001, %eax
  syscall
```

# Conclusion

The 64 bit assembly looks more vague than the tutorials written in X86 assembly. Once you know these basic differences, it's easy for you to learn assembly in depth on your own, even if the material is designed for x86. I highly recommend the book "Programming from the ground up". It is well written for self study purpose.

# References


1. OS X ABI Mach-O File Format Reference
2. System V Application Binary Interface
3. OS X Assembler Reference Assembler Directives
4. Mach-O Programming Topics
5. Mach-O Executables - Build Tools

6. Book: Programming from the ground up.

Posted by dryman (Felix Ren-Chyan Chern) Dec 2nd, 2014 ASM

0 points Tweet

Like Share 20 people like this. Sign Up to see what your friends like.

« Integer Promotion Part 2 Autoconf Tutorial Part-1 »

# Comments

16 Comments Carpe diem 1 Login

Recommend 2 Tweet Share Sort by Best

Join the discussion…

LOG IN WITH OR SIGN UP WITH DISQUS ?

Name

**William Matthews** • 5 months ago • edited

`movq str@GOTPCREL(%rip), %rdi` can be replaced by `leaq str(%rip), %rsi` . Also, you can use `start` instead of `_main` with no need to specify the entry point. You'll also be glad to know that .text and .data directives can be used instead of the more tiresome `.section __TEXT,__text` and `.section __DATA,__data` directives.

1 • Reply • Share ›

**Atul** • 10 months ago • edited

linux, macOS uses System V AMD64 ABI, where
https://en.wikipedia.org/wi...

The first six integer or pointer arguments are passed in registers RDI, RSI, RDX, RCX, R8, R9

And when using system calls, the syscall number must be placed in rax register and when returning register rax contains the result of the system call

1 • Reply • Share ›

**Peter Cordes** • 2 years ago • edited

You don't need to load addresses from the GOT, just use `leaq str(%rip), %rdi`. Also, `xor %eax

You don't need to load addresses from the GOT, just use `lea str(%rip), %rdi`. Also, `xor %eax, %eax` is a better way to zero AL than `movb $0, %al`. (Always at least as fast, and same code size).

You only need to load an address from the GOT if the symbol isn't defined in the executable, i.e. if it comes from a shared library so the distance between the `lea` and the string itself isn't a link-time constant.

1 • Reply • Share ›

**Alex** • 4 years ago

For whatever reason, error code seems to be getting set by %edi for me, rather than %ebx

1 • Reply • Share ›

**sy2002** → Alex • 4 years ago

Same for me: When I want the exit code to be "23", then I need to change the above-mentioned code to for the "exit" example to "
movl $0, %edi". I am comiling and running on El Capitan (OSX 10.11).
Is there a typo in above-mentioned code - or - are there maybe differences between OSX versions?

1 • Reply • Share ›

**jabbathehutt1234** → sy2002 • 3 years ago

I think it's a typo. In the second snippet he uses `edi`.

• Reply • Share ›

**zormpas** • 2 years ago

Ugh, this is assembly language for C programmers. Assembly programmers would NEVER write code this way - too indirect! Lower case assembly source sucks too...

• Reply • Share ›

**Daryl Lee** • 2 years ago

I just discovered this very useful article, and spent an hour on the "exit.s" example before I got it to work on my 10.12.6 laptop. It seems that now (https://stackoverflow.com/q... you have to have a __DATA,__data section.

• Reply • Share ›

**jabbathehutt1234** • 3 years ago • edited

Thanks for the great tutorial. In the first code snippet, I actually get an exit code of 1. Here is the code: https://gist.github.com/ano...

• Reply • Share ›

**jabbathehutt1234** ↪ jabbathehutt1234 • 3 years ago

Never mind, it appears the first parameter to syscalls goes in the `edi` register. Not sure if this is a version thing, but maybe you should update your post.

Reply • Share ›

**Antoine** • 3 years ago

A funny thing with your code (printf one), if the binary filename is longer than 12 characters, I get a segmentation fault... Do you why?

Reply • Share ›

**Ishan Sharma** • 4 years ago

thanks man, great tutorial.
I'm stuck at one problem.
I'm trying to copy a file via syscall 227(0xe3), but I'm getting return value 16 in rax register and copy is not happening, is there anything wrong I'm doing?
Here is the code snippet:

```
start:
pushq %rbp
movq %rsp, %rbp
movq a@GOTPCREL(%rip), %rdi
movq b@GOTPCREL(%rip), %rsi
movq $0x0000000000000000, %rdx
# movq $0x0000000f, %rcx
movq $0x0000000f, %r10
movl $0x20000e3, %rax
syscall

popq %rbp
movl $0x2000001, %eax
syscall
```

Reply • Share ›

**Ian** • 5 years ago

Hi Felix, I found your post on PTT and I just wanna say I'm really motivated by your background and story. I graduated with a EE degree as well but I'm really interested in becoming a developer. I self-taught programming as well and I'm looking forward to pursue a career in Software engineering (or maybe get a CS degree first?)

Anyways, I look forward to see more updates from you! Keep it up!

Reply • Share ›

**Felix Chern** Mod → Ian • 5 years ago

Thanks. If your goal is to become a talented engineer, I think you can create a technical blog like this one, and also work on some open source projects on Github. You can either contribute to good projects, or create your own repository.
I don't have a CS degree, however it doesn't matter when you have the skill. ;)

• Reply • Share ›

**Arete** • 5 years ago

What editor are you using?

• Reply • Share ›

**Felix Chern** Mod → Arete • 5 years ago

I use vim.

1 • Reply • Share ›

ALSO ON **CARPE DIEM**

**Java fast IO using java.nio API - Carpe diem (Felix's blog)**

10 comments • 6 years ago

Duncan — In strategy 2 you should decorate file input stream with a buffered input stream. And you'll see that strategy 2 is not so bad.

**LibSVM on Mac OSX - Carpe diem (Felix's blog)**

7 comments • 7 years ago

Andreu Rivera — Hi, I'll try to make it clearer:On this step "SVM-Toy (GUI)" the author tell you to do this: "Now, goto svm-toy/qt directory.

**Autoconf Tutorial Part-3 - Carpe diem (Felix's blog)**

1 comment • 4 years ago

Petr Bílek — Cant wait for the next post :)

**Convert uf8 literals in Java - Carpe diem (Felix's blog)**

5 comments • 6 years ago

Sky Kelsey — I tried this approach in an Android app. Although it worked, the IBM library added 8.7MB to the download size! In

Subscribe Add Disqus to your siteAdd DisqusAdd Disqus' Privacy PolicyPrivacy PolicyPrivacy

## Recent Posts

- learn hash table the hard way -- part 3: probe distributions and run time performance

- learn hash table the hard way -- part 2: probe distributions with deletions
- learn hash table the hard way -- part 1: probe distributions
- Writing a memory allocator for fast serialization
- Writing a damn fast hash table with tiny memory footprints
- Autoconf Tutorial Part-3
- Autoconf Tutorial Part-2
- Autoconf Tutorial Part-1
- Writing 64 bit assembly on Mac OS X
- Integer Promotion Part 2

## Latest Tweets

- Status updating...

Follow @idryman

Copyright © 2017 - dryman (Felix Ren-Chyan Chern) - Powered by Octopress